



模式识别与计算机视觉

Chapter 3: Filtering

王利民

媒体计算课题组

<http://mcg.nju.edu.cn/>



Overview



- **Image transformation: filtering**
- Image noise and image smoothing
- Convolution operation
- Media filtering
- Bilateral filtering



Image transformations



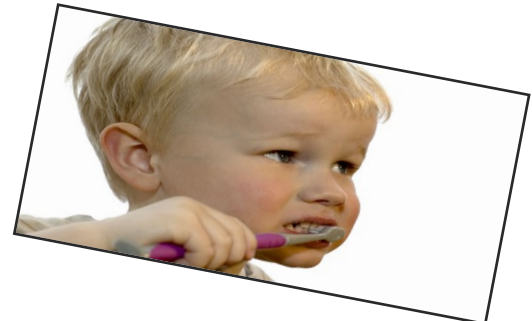
Filtering



changes pixel *values*



Warping



changes pixel *locations*



Image transformations



F



Filtering



$$G(\mathbf{x}) = h\{F(\mathbf{x})\}$$

G



changes *range* of image
function

F

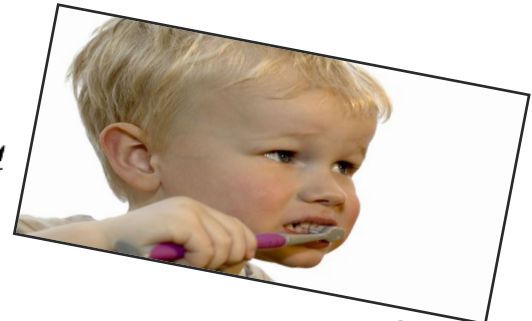


Warping



$$G(\mathbf{x}) = F(h\{\mathbf{x}\})$$

G



changes *domain* of image
function



Examples of point processing



original



darken



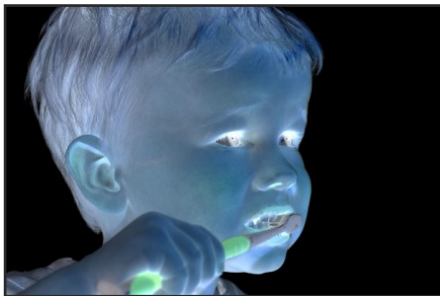
lower contrast



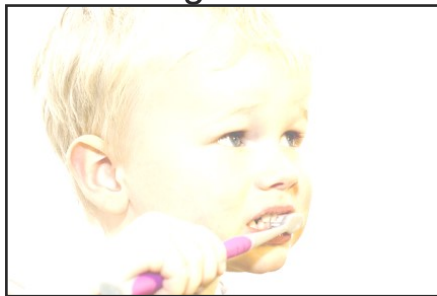
non-linear lower contrast



invert



lighten



raise contrast



non-linear raise contrast





Examples of point processing



How would you implement these?

original



darken



lower contrast

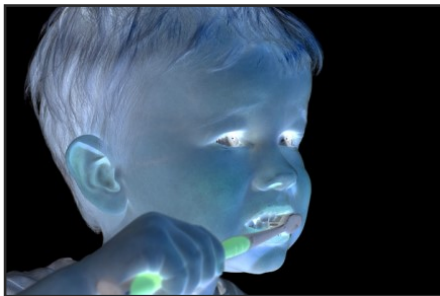


non-linear lower contrast

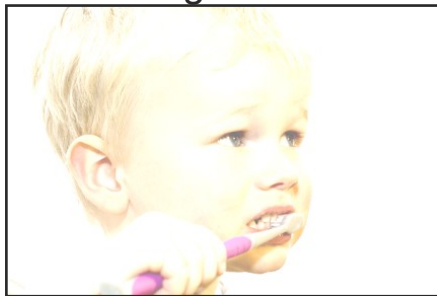


x

invert



lighten



raise contrast



non-linear raise contrast





Examples of point processing



How would you implement these?

original



$$x$$

darken



$$x - 128$$

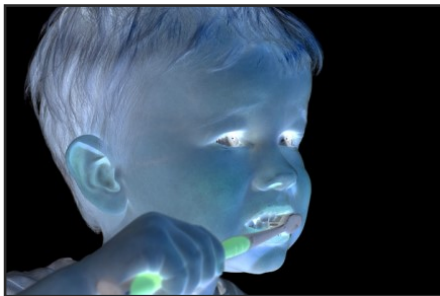
lower contrast



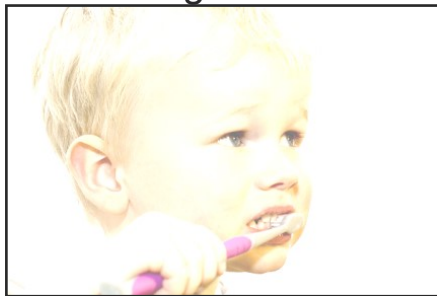
non-linear lower contrast



invert



lighten



raise contrast



non-linear raise contrast





Examples of point processing



How would you implement these?

original



$$x$$

darken



$$x - 128$$

lower contrast

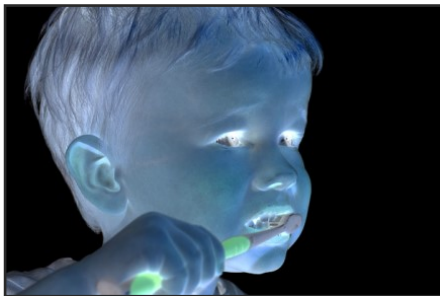


$$\frac{x}{2}$$

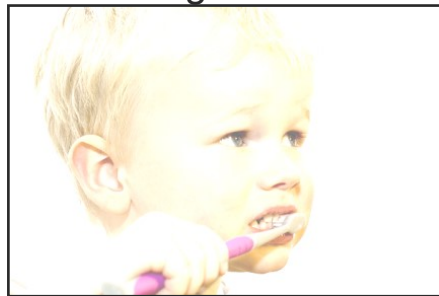
non-linear lower
contrast



invert



lighten



raise contrast



non-linear raise contrast





Examples of point processing



How would you implement these?

original



$$x$$

darken



$$x - 128$$

lower contrast



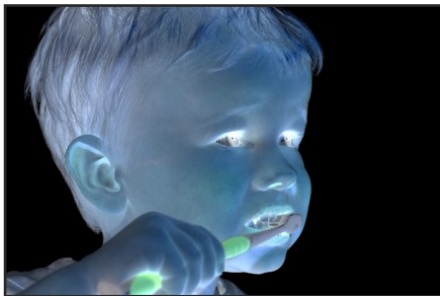
$$\frac{x}{2}$$

non-linear lower contrast

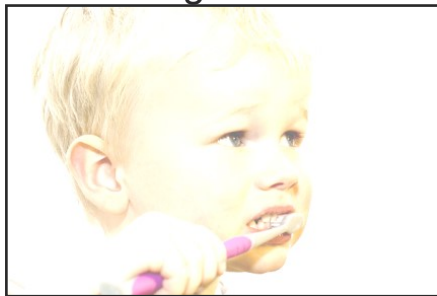


$$\left(\frac{x}{255} \right)^{1/3} \times 255$$

invert



lighten



raise contrast



non-linear raise contrast





Examples of point processing



How would you implement these?

original



$$x$$

darken



$$x - 128$$

lower contrast



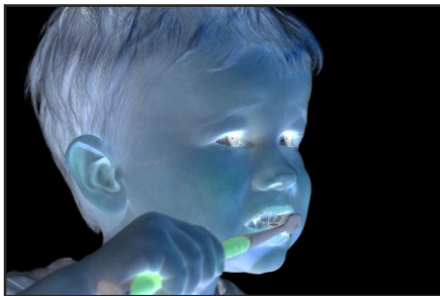
$$\frac{x}{2}$$

non-linear lower
contrast



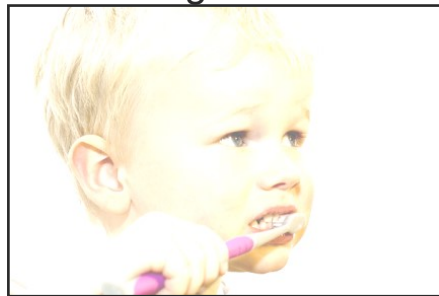
$$\left(\frac{x}{255} \right)^{1/3} \times 255$$

invert



$$255 - x$$

lighten



raise contrast



non-linear raise contrast





Examples of point processing



How would you implement these?

original



$$x$$

darken



$$x - 128$$

lower contrast



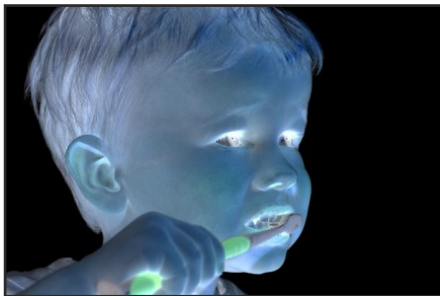
$$\frac{x}{2}$$

non-linear lower contrast



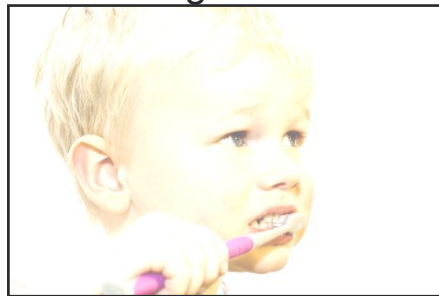
$$\left(\frac{x}{255} \right)^{1/3} \times 255$$

invert



$$255 - x$$

lighten



$$x + 128$$

raise contrast



non-linear raise contrast





Examples of point processing



How would you implement these?

original



$$x$$

darken



$$x - 128$$

lower contrast



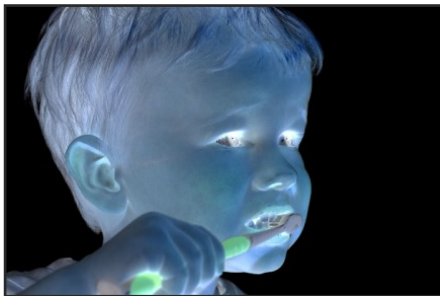
$$\frac{x}{2}$$

non-linear lower
contrast



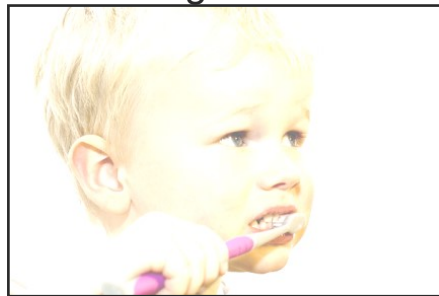
$$\left(\frac{x}{255} \right)^{1/3} \times 255$$

invert



$$255 - x$$

lighten



$$x + 128$$

raise contrast



$$x \times 2$$

non-linear raise contrast





Examples of point processing



How would you implement these?

original



$$x$$

darken



$$x - 128$$

lower contrast



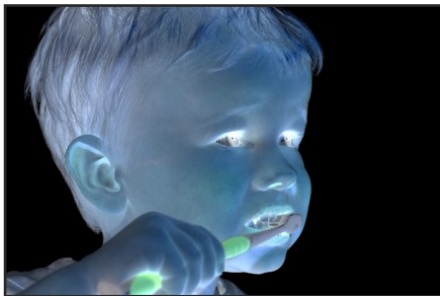
$$\frac{x}{2}$$

non-linear lower contrast



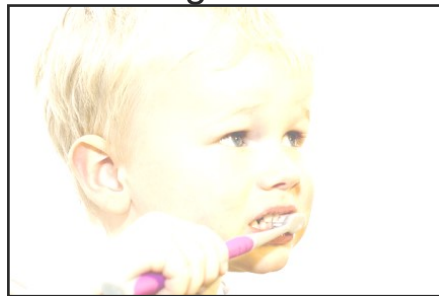
$$\left(\frac{x}{255}\right)^{1/3} \times 255$$

invert



$$255 - x$$

lighten



$$x + 128$$

raise contrast



$$x \times 2$$

non-linear raise contrast



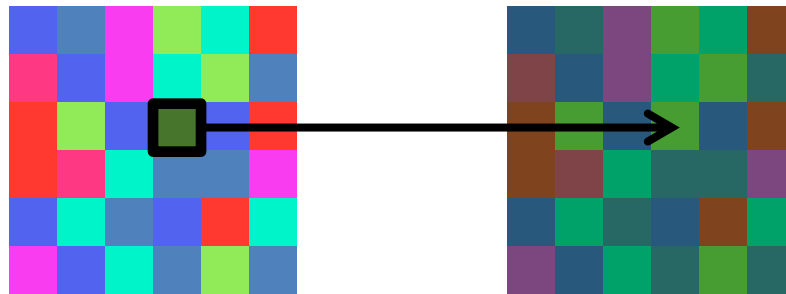
$$\left(\frac{x}{255}\right)^2 \times 255$$



Image filtering

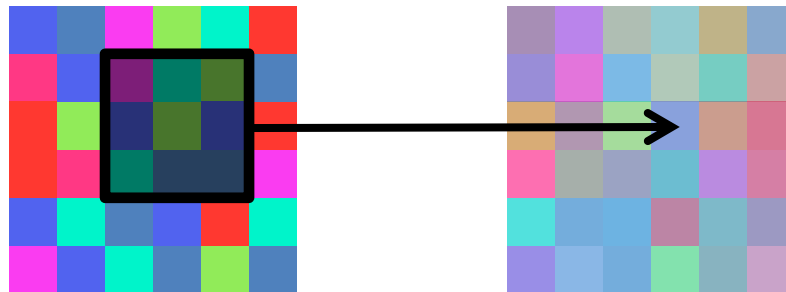


Point Operation



point processing

Neighborhood Operation



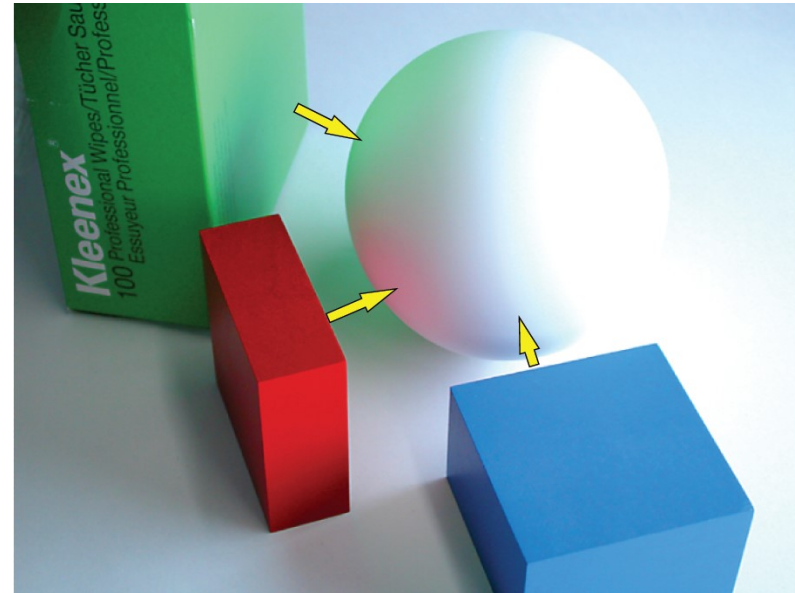
“filtering”



The light of the poor pixel



- A pixel's brightness is determined by
 - Light source (strength, direction, color)
 - Surface orientation
 - Surface material and albedo
 - Reflected light and shadows from surrounding surfaces
 - Gain on the sensor



- A pixel's brightness tells us nothing by itself



And yet we can interpret images...



- Key idea: for nearby scene points, most factors do not change much
- The information is mainly contained in *local differences* of brightness



Image filtering



- Compute a function of the **local neighborhood** at each pixel in the image
 - Function specified by a “filter” or mask saying how to combine values from neighbors.

- Uses of filtering:
 - **Enhance an image** (denoise, resize, etc)
 - Extract information (texture, edges, etc)
 - Detect patterns (template matching)



Three views of filtering



- Image filters in spatial domain
 - Filter is a mathematical operation on values of each patch
 - Smoothing, sharpening, measuring texture
- Image filters in the frequency domain
 - Filtering is a way to modify the frequencies of images
 - Denoising, sampling, image compression
- Templates and Image Pyramids
 - Filtering is a way to match a template to the image
 - Detection, coarse-to-fine registration



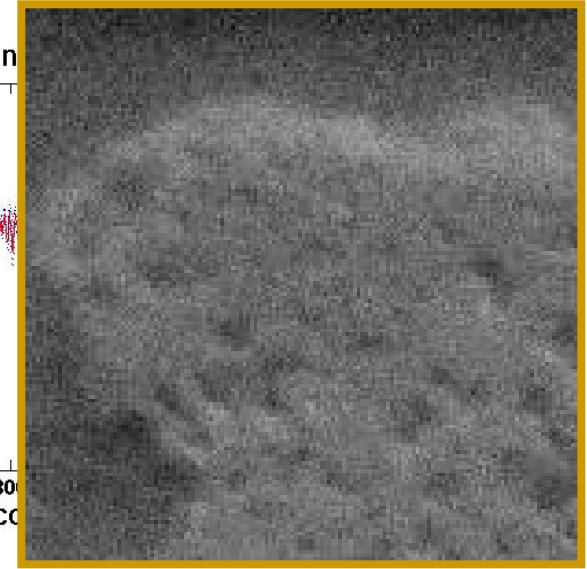
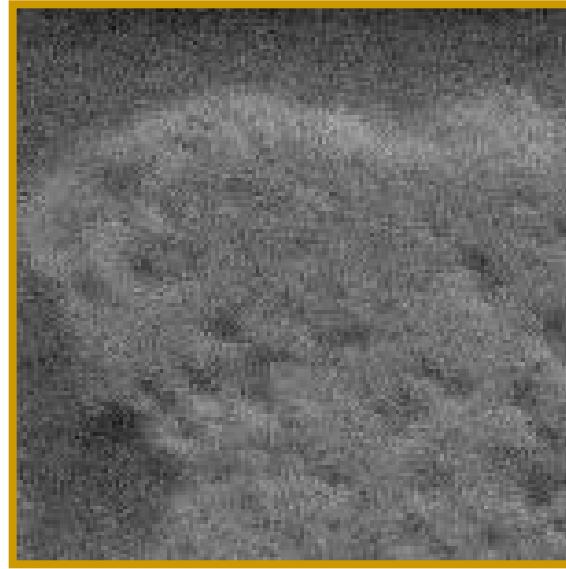
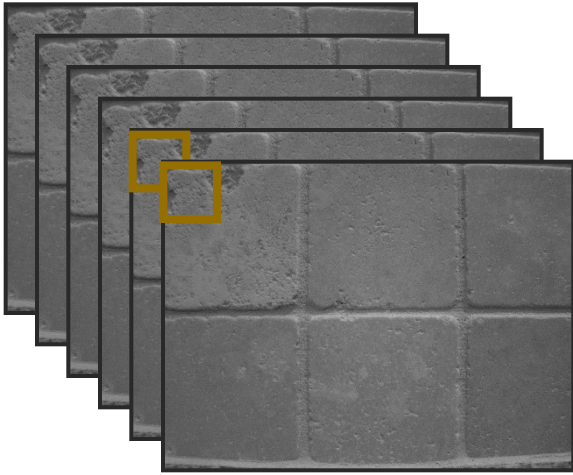
Overview



- Image transformation: filtering
- **Image noise and image smoothing**
- Convolution operation
- Media filtering
- Bilateral filtering



Motivation: noise reduction



- Even multiple images of the **same static scene** will not be identical.



Common types of noise



- **Salt and pepper noise:** random occurrences of black and white pixels
- **Impulse noise:** random occurrences of white pixels
- **Gaussian noise:** variations in intensity drawn from a Gaussian normal distribution



Original



Salt and pepper noise



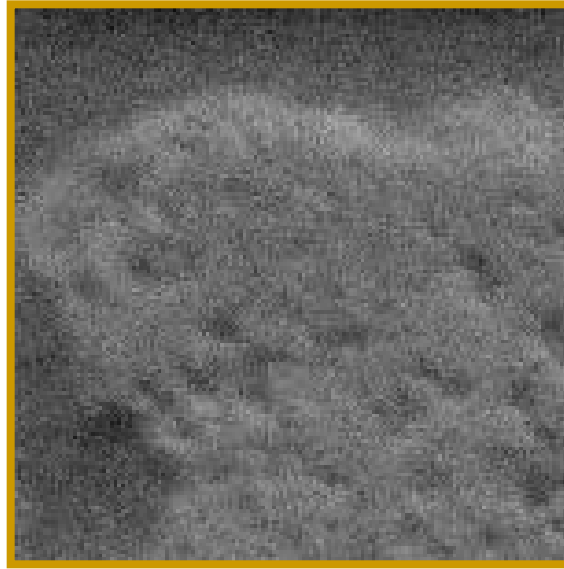
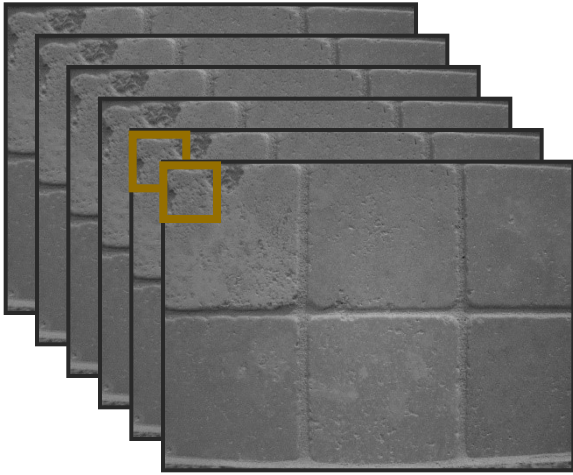
Impulse noise



Gaussian noise



Motivation: noise reduction



- Even multiple images of the same static scene will not be identical.
- How could we reduce the noise, i.e., give an estimate of the true intensities?
- **What if there's only one image?**



First attempt at a solution



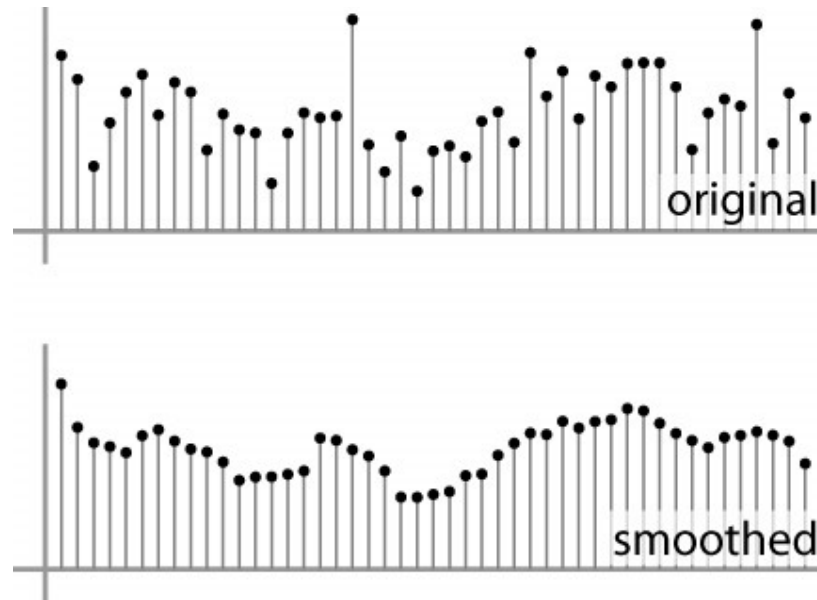
- Let's replace each pixel with an average of all the values in its neighborhood
- Assumptions:
 - Expect pixels to be like their neighbors
 - Expect noise processes to be independent from pixel to pixel



First attempt at a solution



- Let's replace each pixel with an average of all the values in its neighborhood
- Moving average in 1D:

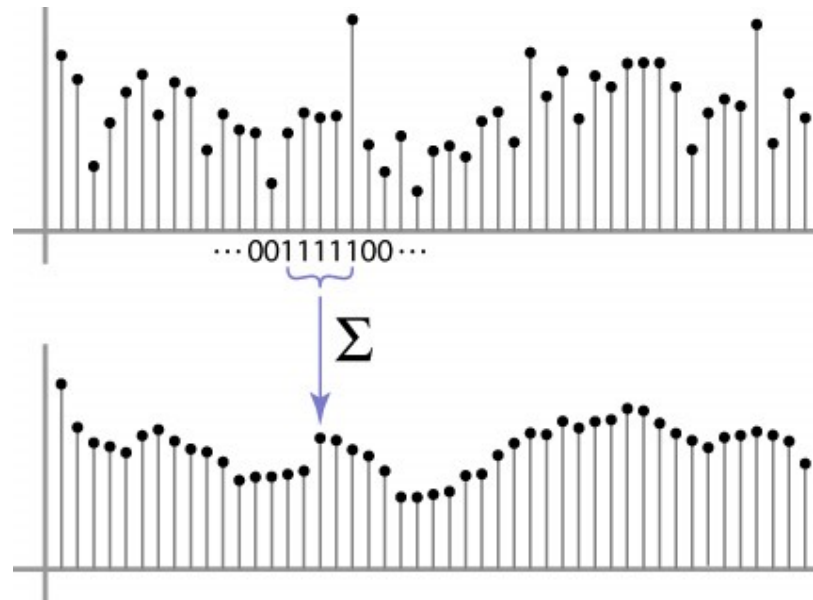




Weighted Moving Average



- Can add weights to our moving average
- *Weights* $[1, 1, 1, 1, 1] / 5$

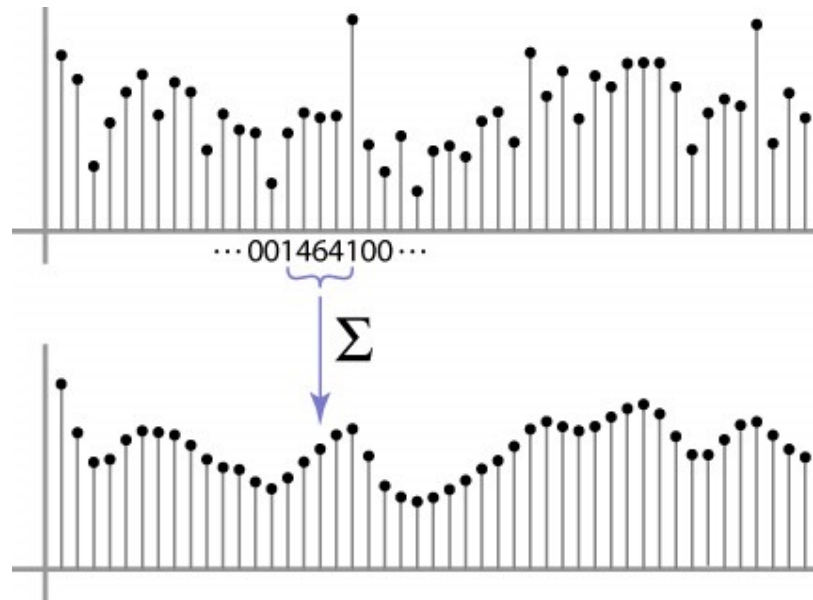


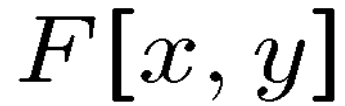


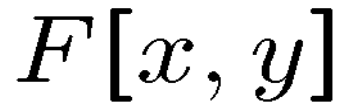
Weighted Moving Average

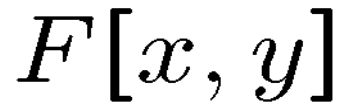


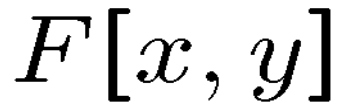
- Non-uniform weights $[1, 4, 6, 4, 1] / 16$

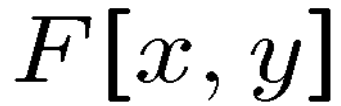


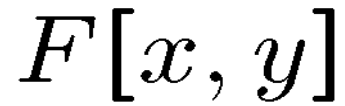

$$G[x, y]$$
[illegible]


$$G[x, y]$$
[illegible]


$$G[x, y]$$
[illegible]


$$G[x, y]$$
[illegible]


$$G[x, y]$$
[illegible]


$$G[x, y]$$
[illegible]



Correlation filtering



Say the averaging window size is $2k+1 \times 2k+1$:

$$G[i, j] = \frac{1}{(2k+1)^2} \sum_{u=-k}^k \sum_{v=-k}^k F[i+u, j+v]$$

*Attribute uniform
weight to each pixel*

*Loop over all pixels in neighborhood
around image pixel $F[i, j]$*

Now generalize to allow different weights depending on neighboring pixel's relative position:

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k \underbrace{H[u, v]}_{\text{Non-uniform weights}} F[i+u, j+v]$$

Non-uniform weights



Correlation filtering



$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v] F[i + u, j + v]$$

This is called cross-correlation, denoted $G = H \otimes F$

Filtering an image: replace each pixel with a linear combination of its neighbors.

The filter “kernel” or “mask” $H[u, v]$ is the prescription for the weights in the linear combination.



Averaging filter



- What values belong in the kernel H for the moving average example?

$F[x, y]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0



$H[u, v]$

$\frac{1}{9}$

1	1	1
1	1	?
1	1	1

“box filter”

$G[x, y]$

	0	20	40	60	60				

$$G = H \otimes F$$



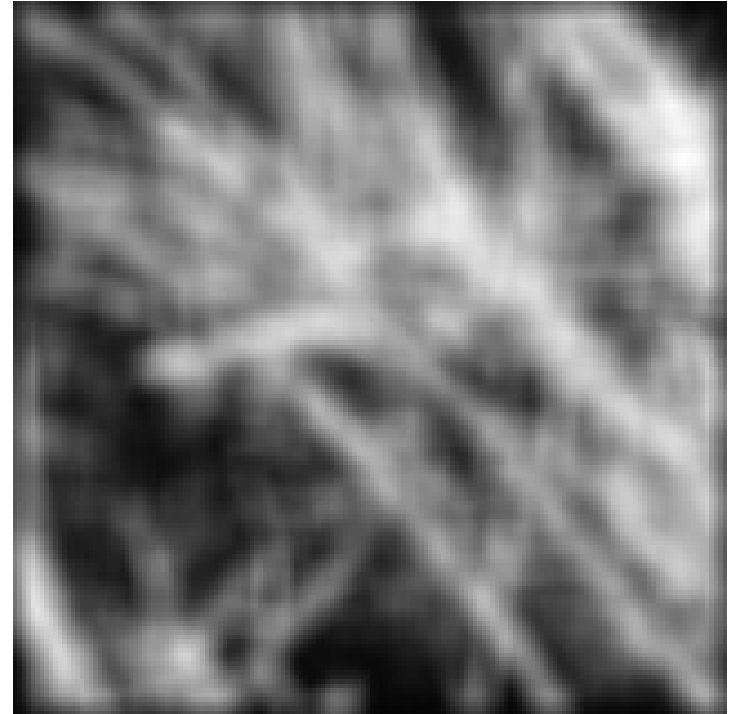
Smoothing by averaging



depicts box filter:
white = high value, black = low value



original



filtered

What if the filter size was 5 x 5 instead of 3 x 3?



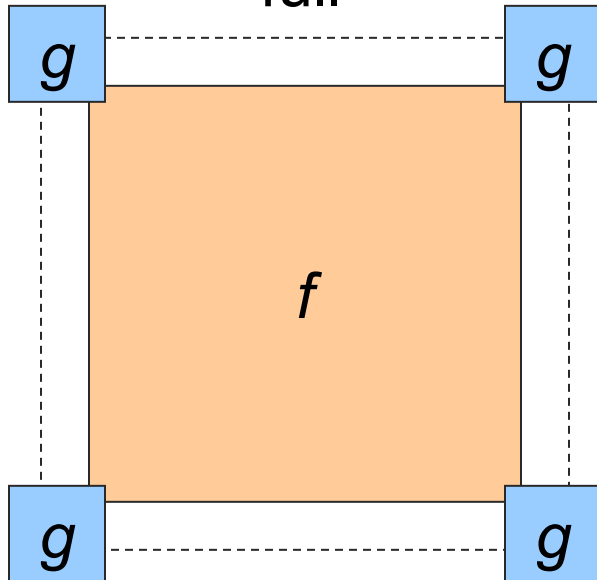
Boundary issues



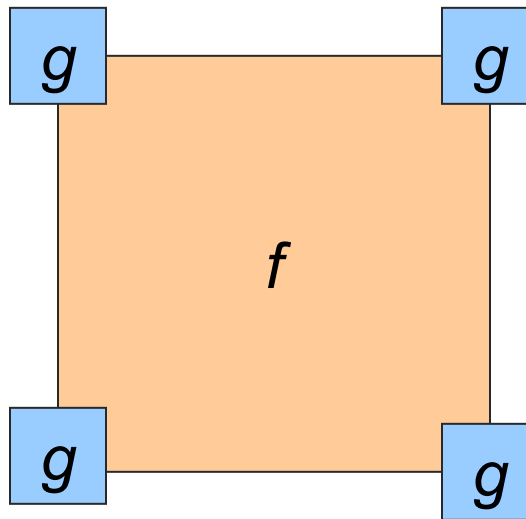
MATLAB: output size / “shape” options

- *shape* = ‘full’: output size is sum of sizes of *f* and *g*
- *shape* = ‘same’: output size is same as *f*
- *shape* = ‘valid’: output size is difference of sizes of *f* and *g*

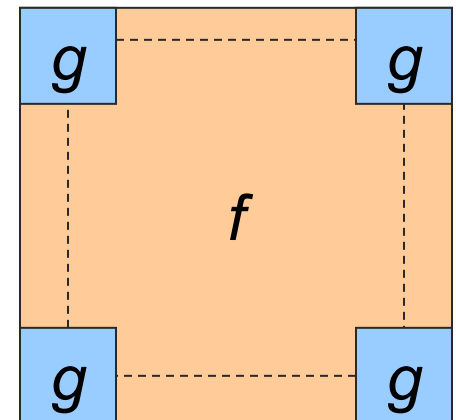
full



same



valid



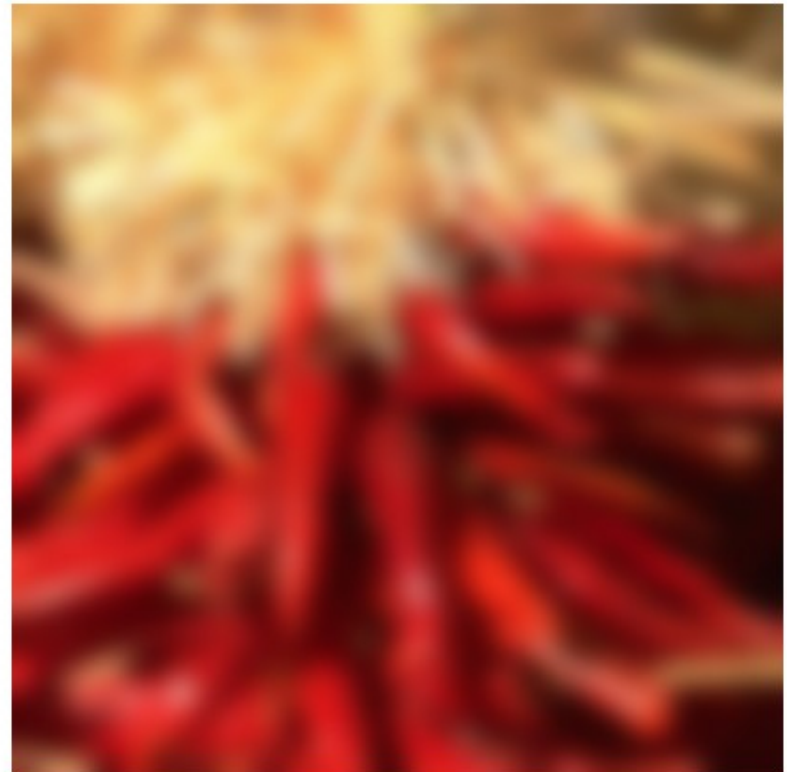
Source: S. Lazebnik



Boundary issues



- What about near the edge?
 - the filter window falls off the edge of the image
 - need to extrapolate
 - methods:
 - clip filter (black)
 - wrap around
 - copy edge
 - reflect across edge





Boundary issues



- What about near the edge?
 - the filter window falls off the edge of the image
 - need to extrapolate
 - methods (MATLAB):
 - clip filter (black): `imfilter(f, g, 0)`
 - wrap around: `imfilter(f, g, 'circular')`
 - copy edge: `imfilter(f, g, 'replicate')`
 - reflect across edge: `imfilter(f, g, 'symmetric')`



Gaussian filter



- What if we want nearest neighboring pixels to have the most influence on the output?

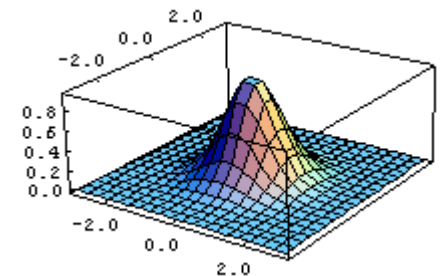
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$F[x, y]$

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} H[u, v]$$

This kernel is an approximation of a 2d Gaussian function:

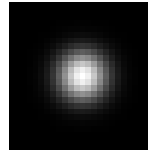
$$h(u, v) = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{\sigma^2}}$$



- Removes high-frequency components from the image (“low-pass filter”).



Smoothing with a Gaussian

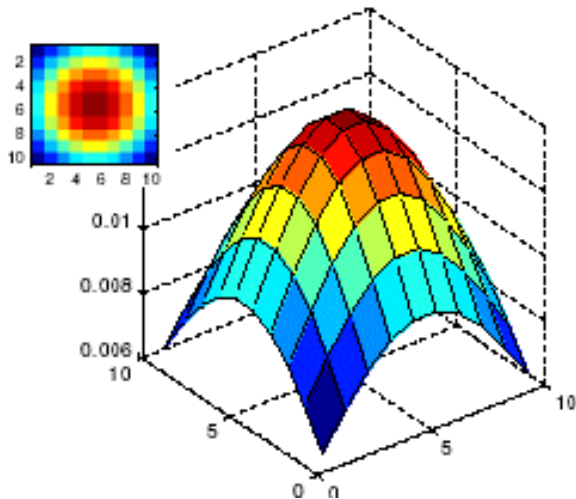




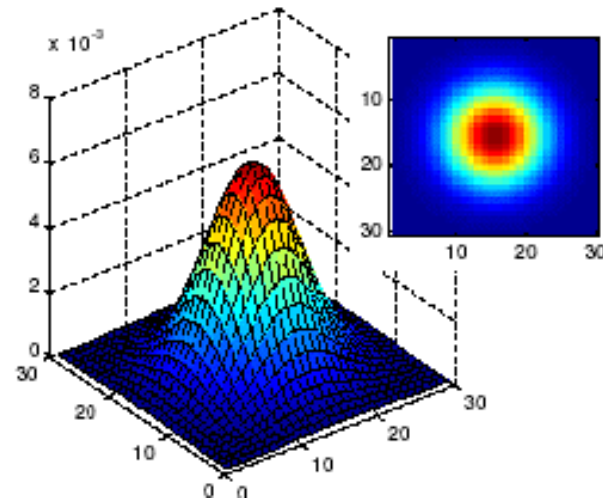
Gaussian filters



- What parameters matter here?
- **Size** of kernel or mask
 - Note, Gaussian function has infinite support, but discrete filters use finite kernels



$\sigma = 5$ with 10×10 kernel



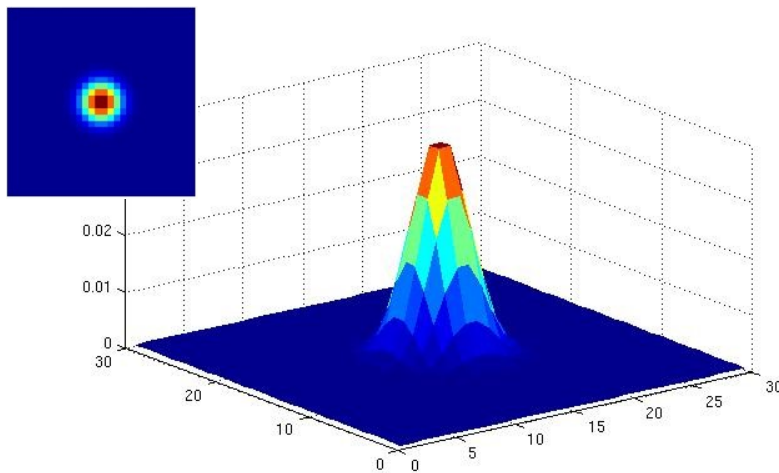
$\sigma = 5$ with 30×30 kernel



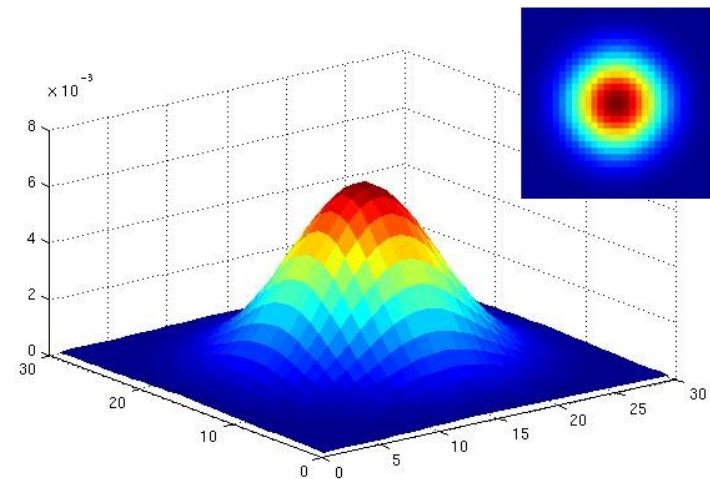
Gaussian filters



- What parameters matter here?
- **Variance** of Gaussian: determines extent of smoothing



$\sigma = 2$ with 30×30 kernel



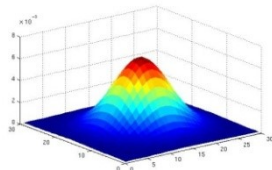
$\sigma = 5$ with 30×30 kernel



Matlab



```
>> hsize = 10;  
>> sigma = 5;  
>> h = fspecial('gaussian' hsize, sigma);
```



```
>> mesh(h);
```



```
>> imagesc(h);
```



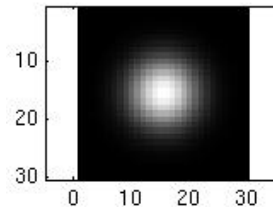
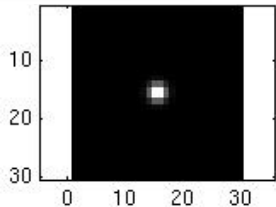
```
>> outim = imfilter(im, h); % correlation outim  
>> imshow(outim);
```



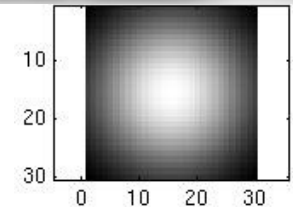
Smoothing with a Gaussian



Parameter σ is the “scale” / “width” / “spread” of the Gaussian kernel, and controls the amount of smoothing.



...



```
for sigma=1:3:10
    h = fspecial('gaussian', fsize,
        sigma);
    out = imfilter(im, h);
    imshow(out);
    pause;
end
```



Properties of smoothing filters



■ Smoothing

- Values positive
- Sum to 1 $\rightarrow$ constant regions same as input
- Amount of smoothing proportional to mask size
- Remove “high-frequency” components; “low-pass” filter



Overview



- Image transformation: filtering
- Image noise and image smoothing
- **Convolution operation**
- Media filtering
- Bilateral filtering



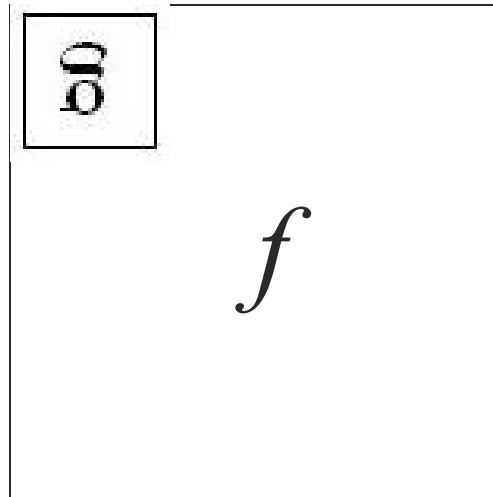
Defining convolution



- Let f be the image and g be the kernel. The output of convolving f with g is denoted $f * g$.

$$(f * g)[m, n] = \sum_{k, l} f[m - k, n - l] g[k, l]$$

Convention:
kernel is “flipped”

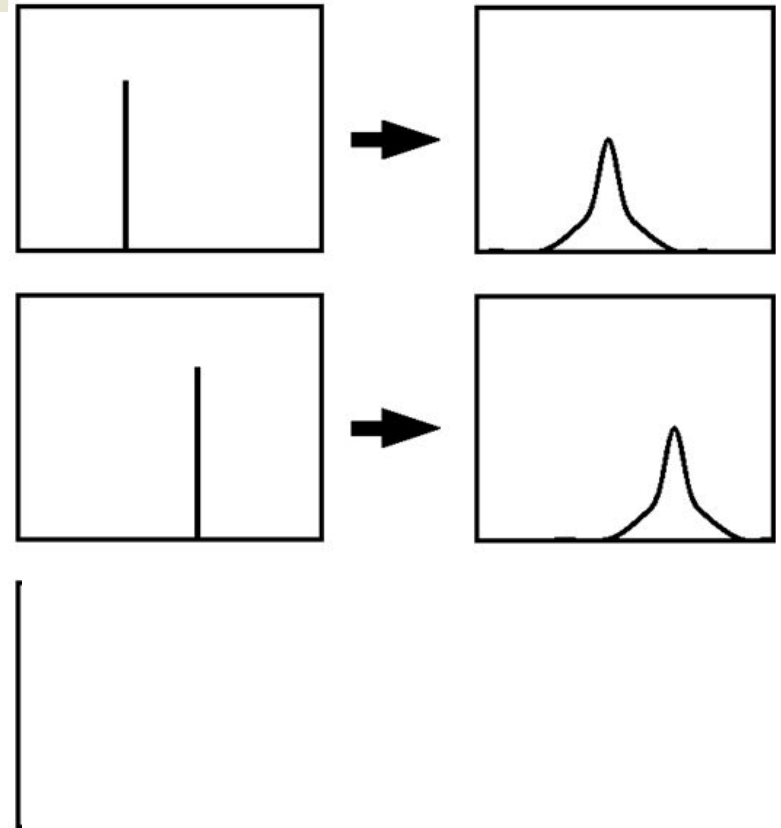




Key properties



- **Shift invariance:** same behavior regardless of pixel location:
 $\text{filter}(\text{shift}(f)) = \text{shift}(\text{filter}(f))$
- **Linearity:**
 $\text{filter}(f_1 + f_2) = \text{filter}(f_1) + \text{filter}(f_2)$



- Theoretical result: any linear shift-invariant operator can be represented as a convolution



Properties in more detail



- Commutative: $a * b = b * a$
 - Conceptually no difference between filter and signal
- Associative: $a * (b * c) = (a * b) * c$
 - Often apply several filters one after another: $((a * b_1) * b_2) * b_3$
 - This is equivalent to applying one filter: $a * (b_1 * b_2 * b_3)$
- Distributes over addition: $a * (b + c) = (a * b) + (a * c)$
- Scalars factor out: $ka * b = a * kb = k(a * b)$
- Identity: unit impulse $e = [..., 0, 0, 1, 0, 0, ...]$,
 $a * e = a$



Convolution vs correlation



Definition of discrete 2D
convolution:

$$(f * g)(x, y) = \sum_{i, j = -\infty}^{\infty} f(i, j) I(x - i, y - j)$$

notice the flip

Definition of discrete 2D
correlation:

$$(f * g)(x, y) = \sum_{i, j = -\infty}^{\infty} f(i, j) I(x + i, y + j)$$

notice the lack of a flip

- Most of the time won't matter, because our kernels will be symmetric.



Separability



- In some cases, filter is separable, and we can factor into two steps:
 - Convolve all rows with a 1D filter
 - Convolve all columns with a 1D filter

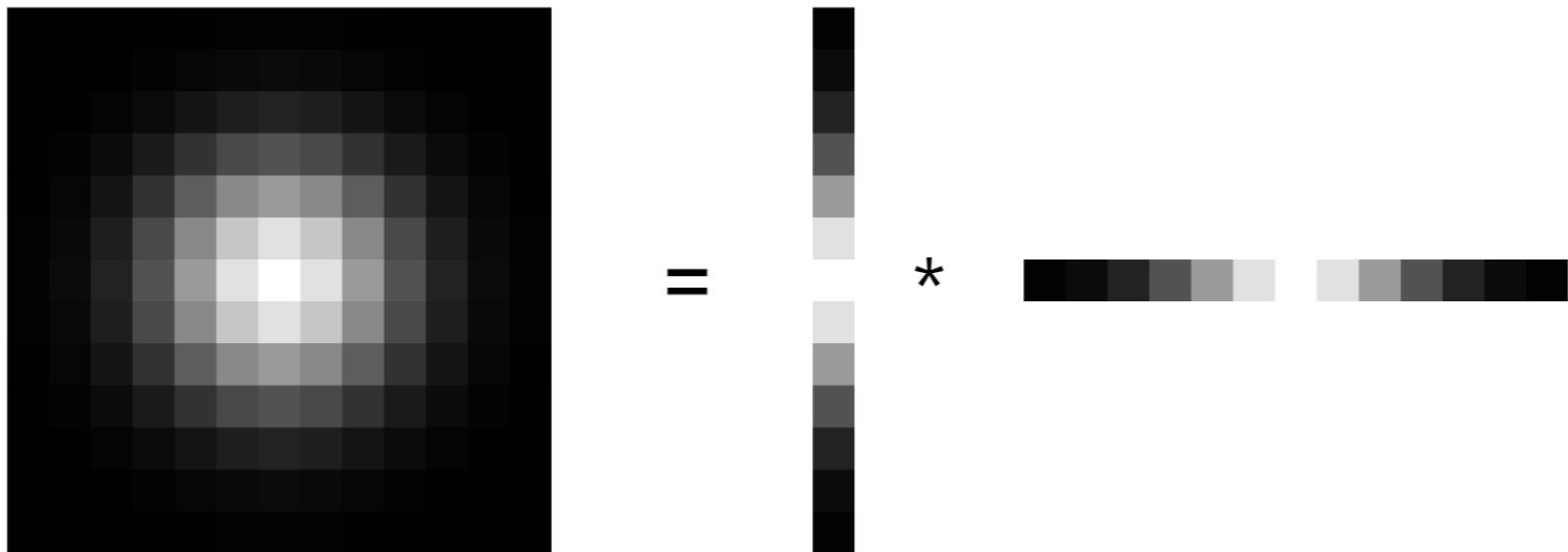
$$\begin{bmatrix} \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \\ \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \\ \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \end{bmatrix} = \begin{bmatrix} 0 & \frac{1}{3} & 0 \\ 0 & \frac{1}{3} & 0 \\ 0 & \frac{1}{3} & 0 \end{bmatrix} \circ \begin{bmatrix} 0 & 0 & 0 \\ \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \\ 0 & 0 & 0 \end{bmatrix}$$



Separability of Gaussian Filter



$$\frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{x^2}{2\sigma^2}\right) \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{y^2}{2\sigma^2}\right)$$





Why separability?



- Separability means that a 2D convolution can be reduced to two 1D convolutions (one along rows and one along columns)
- What is the complexity of filtering an $n \times n$ image with an $m \times m$ kernel?
 - $O(n^2 m^2)$
- What if the kernel is separable?
 - $O(n^2 m)$



Overview



- Image transformation: filtering
- Image noise and image smoothing
- Convolution operation
- **Media filtering**
- Bilateral filtering



Effect of smoothing filters



5x5



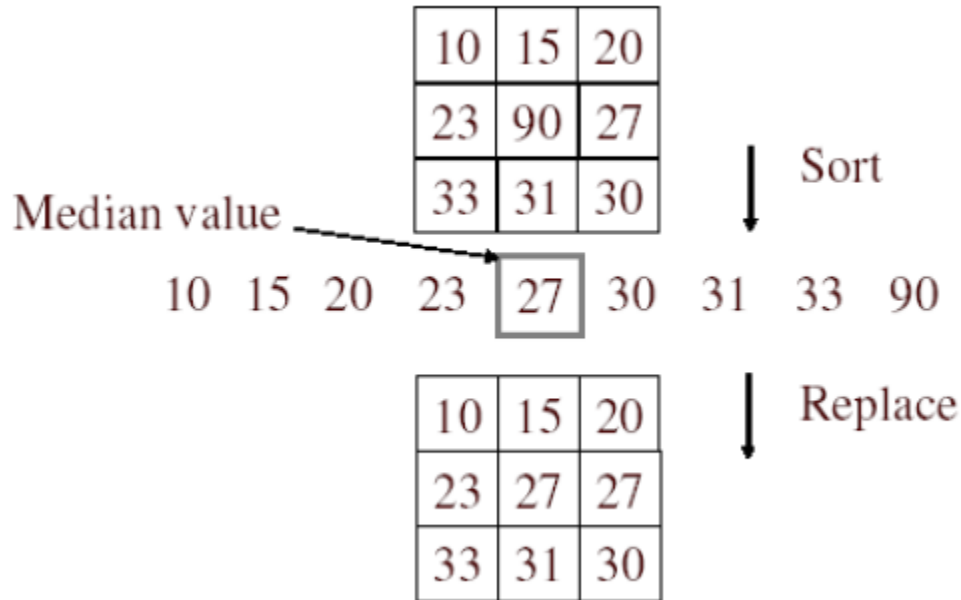
Additive Gaussian noise



Salt and pepper noise



Median filter



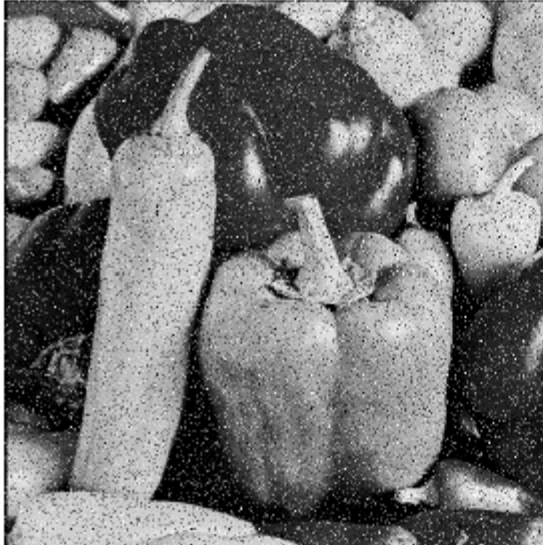
- No new pixel values introduced
- Removes spikes: good for impulse, salt & pepper noise
- Non-linear filter



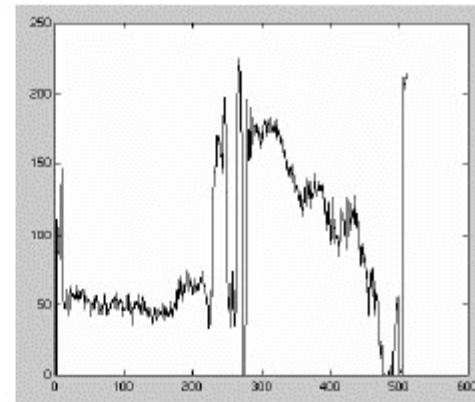
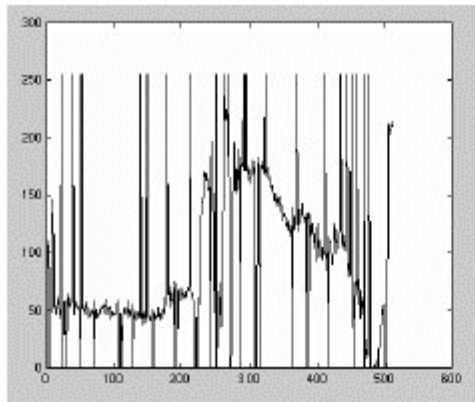
Median filter



Salt and
pepper
noise



Median
filtered



Plots of a row of the image

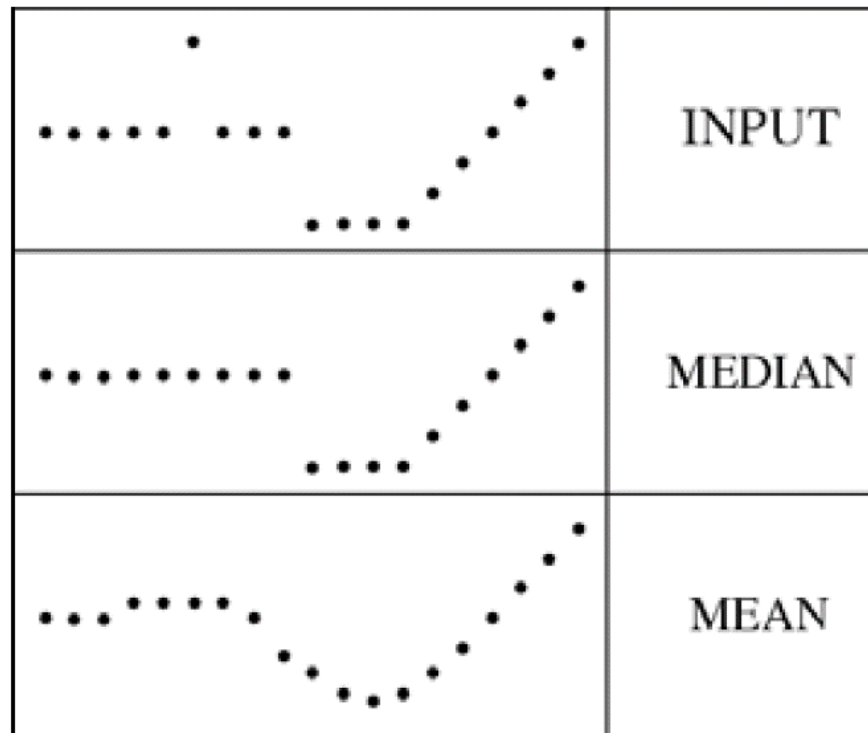
Matlab: `output im = medfilt2(im, [h w]);`



Median filter



- Median filter is edge preserving





Is median filtering linear?



$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 2 \\ 2 & 2 & 2 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 2 \\ 2 & 2 & 2 \end{bmatrix}$$

median
filter



1

+



0

≠



2



Overview



- Image transformation: filtering
- Image noise and image smoothing
- Convolution operation
- Media filtering
- **Bilateral filtering**



Objective of bilateral filtering



Smooth texture

Preserve edges



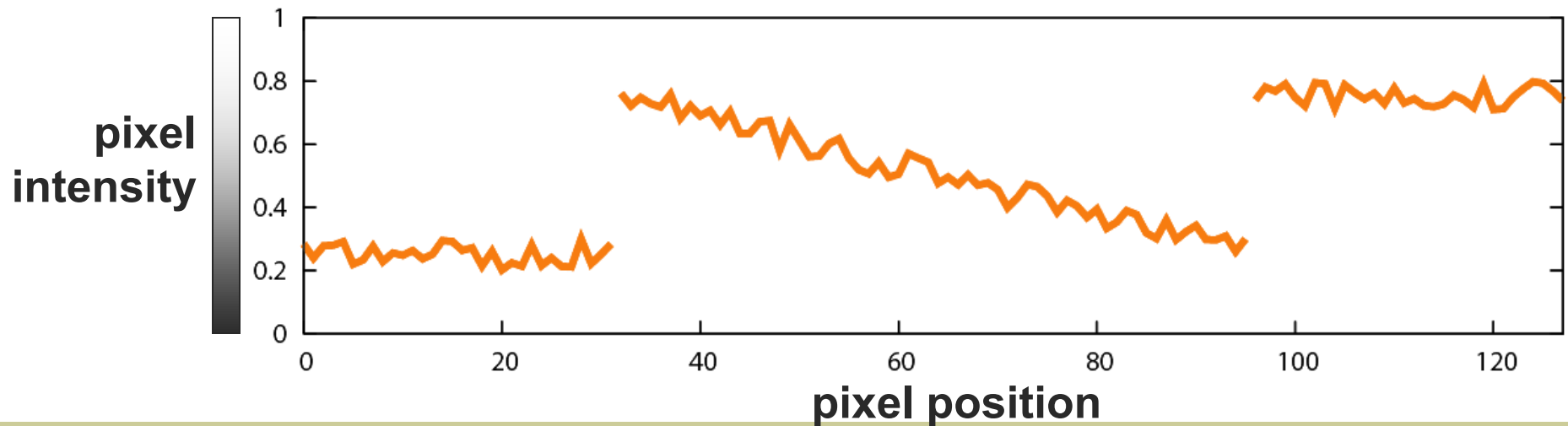
Illustration a 1D Image



- 1D image = line of pixels



- Better visualized as a plot





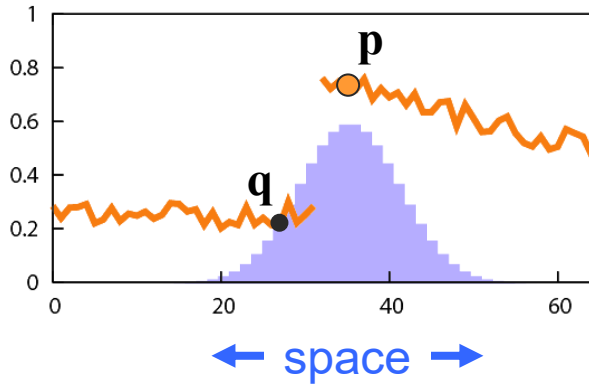
Definition



Gaussian blur

$$I_{\mathbf{p}}^b = \sum_{\mathbf{q} \in \mathcal{S}} \underbrace{G_{\sigma_s}(\|\mathbf{p} - \mathbf{q}\|)}_{\text{space}} I_{\mathbf{q}}$$

- only spatial distance, intensity ignored

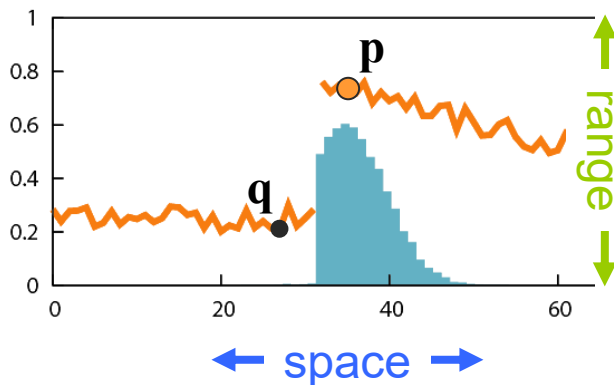


Bilateral filter

[Aurich 95, Smith 97, Tomasi 98]

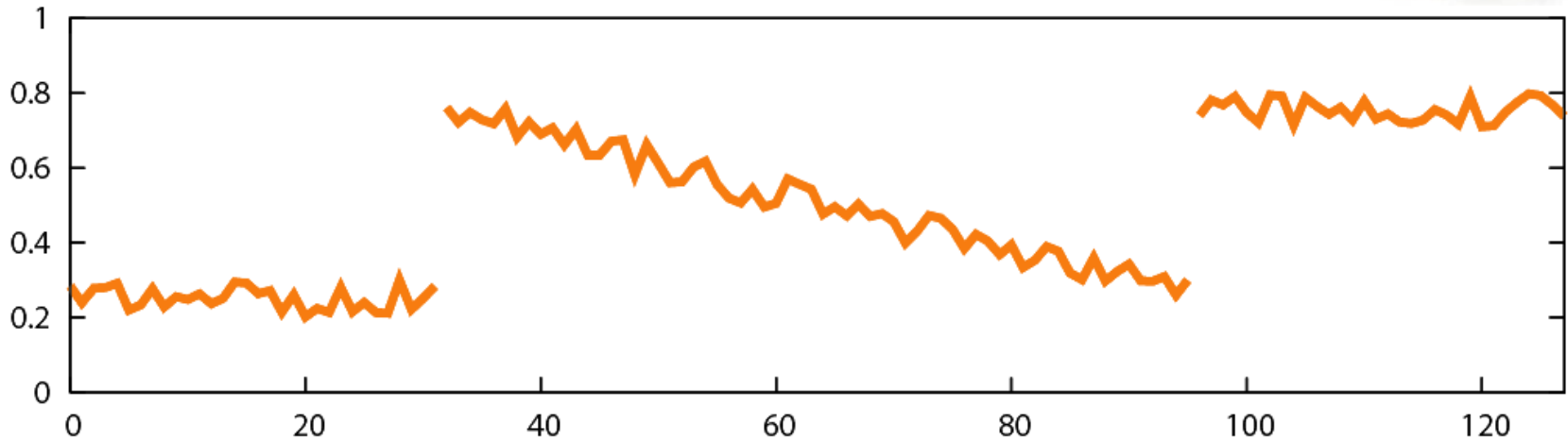
$$I_{\mathbf{p}}^{\text{bf}} = \underbrace{\frac{1}{W_{\mathbf{p}}^{\text{bf}}}}_{\text{normalization}} \sum_{\mathbf{q} \in \mathcal{S}} \underbrace{G_{\sigma_s}(\|\mathbf{p} - \mathbf{q}\|)}_{\text{space}} \underbrace{G_{\sigma_r}(|I_{\mathbf{p}} - I_{\mathbf{q}}|)}_{\text{range}} I_{\mathbf{q}}$$

- spatial and range distances
- weights sum to 1

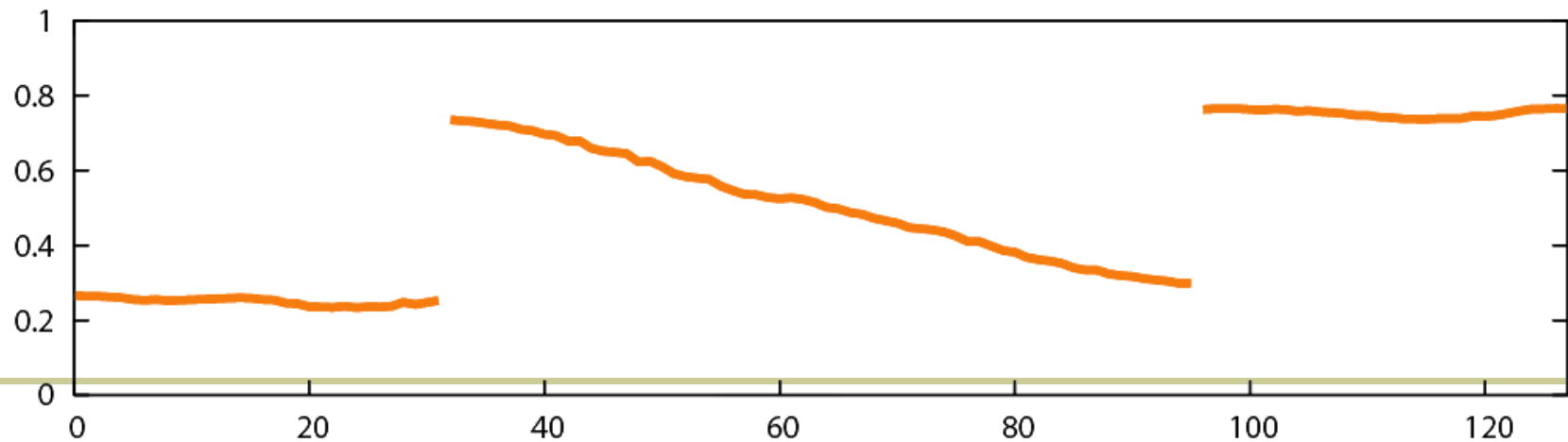




Bilateral Filter on 1D Signal



BF



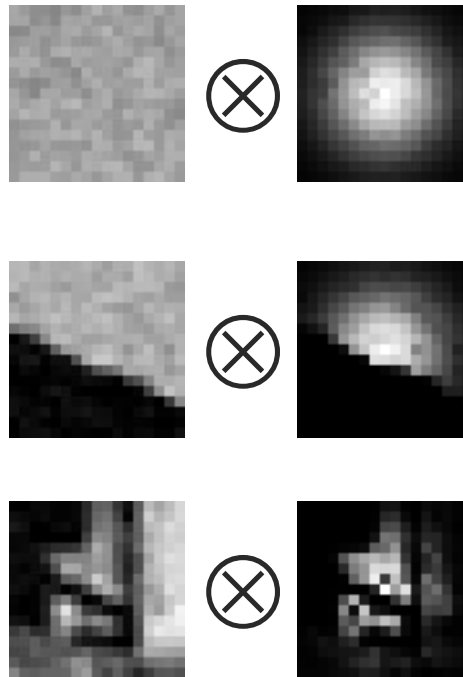
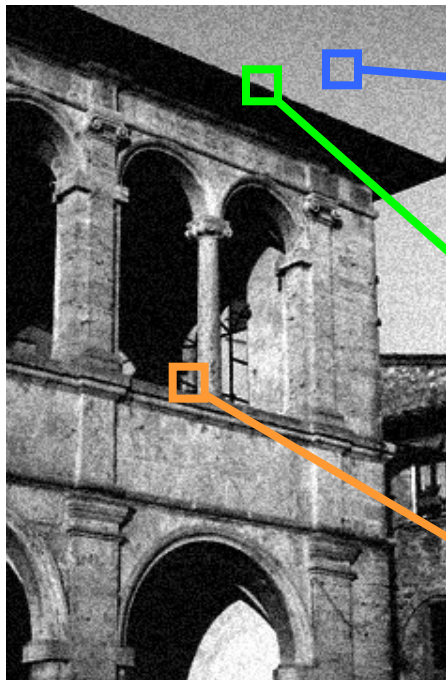


Example on a Real Image



- Kernels can have complex, spatially varying shapes.

input



output





Bilateral Filter is Expensive



- Brute-force computation is slow (several minutes)
 - Two nested for loops:
for each pixel, look at all pixels
 - Non-linear, depends on image content
⇒ no FFT, no pre-computation...
- Fast approximations exist [Durand 02, Weiss 06]
 - Significant **loss of accuracy**
 - **No formal understanding** of accuracy versus speed



(a)



(b)



(c)



(d)



(e)



(f)



(g)



(h)

Figure 3.18 Median and bilateral filtering: (a) original image with Gaussian noise; (b) Gaussian filtered; (c) median filtered; (d) bilaterally filtered; (e) original image with shot noise; (f) Gaussian filtered; (g) median filtered; (h) bilaterally filtered. Note that the bilateral filter fails to remove the shot noise because the noisy pixels are too different from their neighbors.



Summary



- Image transform: filtering
 - Point processing, Neighborhood operation
- Image noise and image smoothing
 - Linear filters: Box filter, Gaussian filter
- Convolution operation
 - Convolution vs. Correlation
- Non-linear filters:
 - Media filter, Bilateral Filter